Handoff: TPV con carta QR

Oct 26, 2026 · @Alonso

Especificación para construir un TPV de restaurante con carta digital por QR, comandas en tiempo real y un registro histórico que nunca se borra. Se entrega en 6 fases; cada una termina con algo usable y probado antes de empezar la siguiente.

Alcance

La versión 1 sirve a un solo restaurante: carta, mesas con QR, pedidos del cliente, comandas en cocina, cobro y un histórico permanente. Multi-local y stock quedan para la fase 5.

Entra en la versión 1

- Panel de administración: categorías, productos, precios, IVA, ingredientes, alérgenos, modificadores, fotos y disponibilidad.
- Mesas: alta, baja, zonas (sala, terraza) y un QR imprimible por mesa.
- Carta pública por QR: navegar, añadir al carrito, notas, modificadores y enviar el pedido.
- Sesión de mesa: el camarero la abre, el cliente pide con un PIN y el camarero la cierra al cobrar.
- Comandas en tiempo real en el panel y en la pantalla de cocina/barra, con estados.
- Cuenta por mesa con total acumulado, cobro (efectivo, tarjeta, mixto), división de la cuenta y ticket.
- Cierre de caja diario e histórico consultable y exportable.

No entra (por ahora)

- Pago online desde el móvil del cliente (fase 5; exige pasarela y PCI).
- Reservas, fidelización y delivery.
- Control de stock por ingrediente (fase 5).

Principios que no se negocian

1. **Nada se borra.** Productos, mesas y usuarios se desactivan. Comandas, líneas y pagos solo se anulan con motivo, usuario y hora.
2. **El histórico es una foto.** Cada línea guarda nombre, precio, IVA y modificadores tal como estaban al pedir.
3. **El servidor manda.** El precio y el total se calculan siempre en el servidor; lo que envía el móvil del cliente solo son IDs y cantidades.

4. **Dinero en céntimos enteros.** Nunca `float`. Redondeo de IVA por línea y documentado.
5. **Hora en UTC en la base de datos**, mostrada en `Europe/Madrid`. El "día de negocio" lo define el cierre de caja, no la medianoche.

Roles y permisos

Hay cuatro roles de personal y un cliente anónimo. Los permisos se comprueban en la base de datos (políticas RLS), no solo ocultando botones en la interfaz.

Acción	Admin	Encargado	Camarero	Cocina/Barra	Cliente (QR)
Editar carta, precios, IVA	Sí	No	No	No	No
Marcar producto agotado	Sí	Sí	Sí	Sí	No
Crear/desactivar mesas y regenerar QR	Sí	Sí	No	No	No
Abrir/cerrar sesión de mesa	Sí	Sí	Sí	No	No
Pedir en una mesa	Sí	Sí	Sí	No	Solo en su sesión abierta
Cambiar estado de comanda	Sí	Sí	Servido	Preparando, listo	No
Anular línea ya enviada	Sí	Sí	Con PIN de encargado	No	No
Descuento o invitación	Sí	Sí (hasta un límite)	No	No	No
Cobrar y emitir ticket	Sí	Sí	Sí	No	No
Reabrir cuenta cobrada	Sí	No	No	No	No
Cierre de caja	Sí	Sí	No	No	No
Ver histórico e informes	Sí	Solo su turno	No	No	No
Gestionar usuarios	Sí	No	No	No	No

El personal entra con email y contraseña (admin y encargado, con 2FA obligatorio) o con un PIN de 4–6 dígitos en un dispositivo ya registrado del local (camarero y cocina). El cliente nunca tiene cuenta: su acceso es un token de sesión de mesa (ver Seguridad).

Flujos

Una mesa es permanente; una sesión de mesa es cada grupo de clientes que se sienta. Todo pedido, pago y registro cuelga de la sesión, no de la mesa.

A. Alta de mesa (admin)

1. El admin crea la mesa: número, nombre, zona y capacidad.
2. El sistema genera un `qr_slug` aleatorio de 22 caracteres (no el número de mesa) y la URL `https://<dominio>/m/<qr_slug>`.
3. El admin descarga el QR en PNG/SVG o imprime una hoja A4 con todos los QR y el nombre de cada mesa debajo.
4. Botón "Regenerar QR": crea un slug nuevo e invalida el anterior al instante (por si roban o fotografían la pegatina).

B. Llegada de clientes (camarero)

1. El camarero toca la mesa en el plano y pulsa "Abrir". Indica número de comensales.
2. Se crea la sesión con un PIN de 4 dígitos que aparece en su pantalla. Lo dice en voz alta o lo escribe en la tarjeta de la mesa.
3. Si la mesa ya tiene sesión abierta, no se puede abrir otra (restricción en base de datos).

C. Pedido del cliente

1. El cliente escanea el QR. Si no hay sesión abierta, ve la carta en modo solo lectura con el aviso "Pide al camarero que abra la mesa".
2. Si hay sesión, introduce el PIN una vez. El servidor le devuelve un token de dispositivo ligado a esa sesión, guardado en una cookie `HttpOnly`.
3. Navega por categorías, ve fotos, alérgenos y precios. Productos agotados aparecen grises y no se pueden añadir.
4. Añade productos con modificadores ("sin cebolla", "punto: al punto", "+ queso 1,00 €") y una nota libre de hasta 140 caracteres.
5. Pulsa "Pedir", revisa el resumen y confirma. La app envía solo `{producto_id, cantidad, modificador_ids, nota}` más una `idempotency_key`.
6. El servidor valida disponibilidad, recalcula precios, crea la comanda y sus líneas en una sola transacción y la emite en tiempo real.
7. El cliente ve "Pedido recibido" y el estado de cada plato. Todos los móviles de la misma mesa ven el mismo historial de pedidos de la sesión.
8. Botones siempre visibles: "Llamar al camarero" y "Pedir la cuenta".

D. Cocina y servicio

1. La comanda aparece en el panel y en la pantalla de cocina/barra con sonido. Cada línea va a la estación de su categoría (bebidas → barra, platos → cocina).
2. Cocina marca "Preparando" y luego "Listo". El camarero marca "Servido".
3. Opción de configuración: "Las comandas del cliente requieren aceptación del camarero" (útil al principio para evitar bromas).

E. Cuenta y cobro

1. El panel de la mesa muestra todas las comandas de la sesión y el total acumulado en vivo.
2. El camarero pulsa "Cobrar": elige pago completo, a partes iguales (N personas) o por productos.
3. Registra uno o varios pagos (efectivo con cambio calculado, tarjeta, mixto) y propina opcional.
4. Cuando lo pagado iguala el total, se emite el ticket (factura simplificada), la sesión pasa a "cerrada", el token del cliente deja de valer y la mesa queda libre.

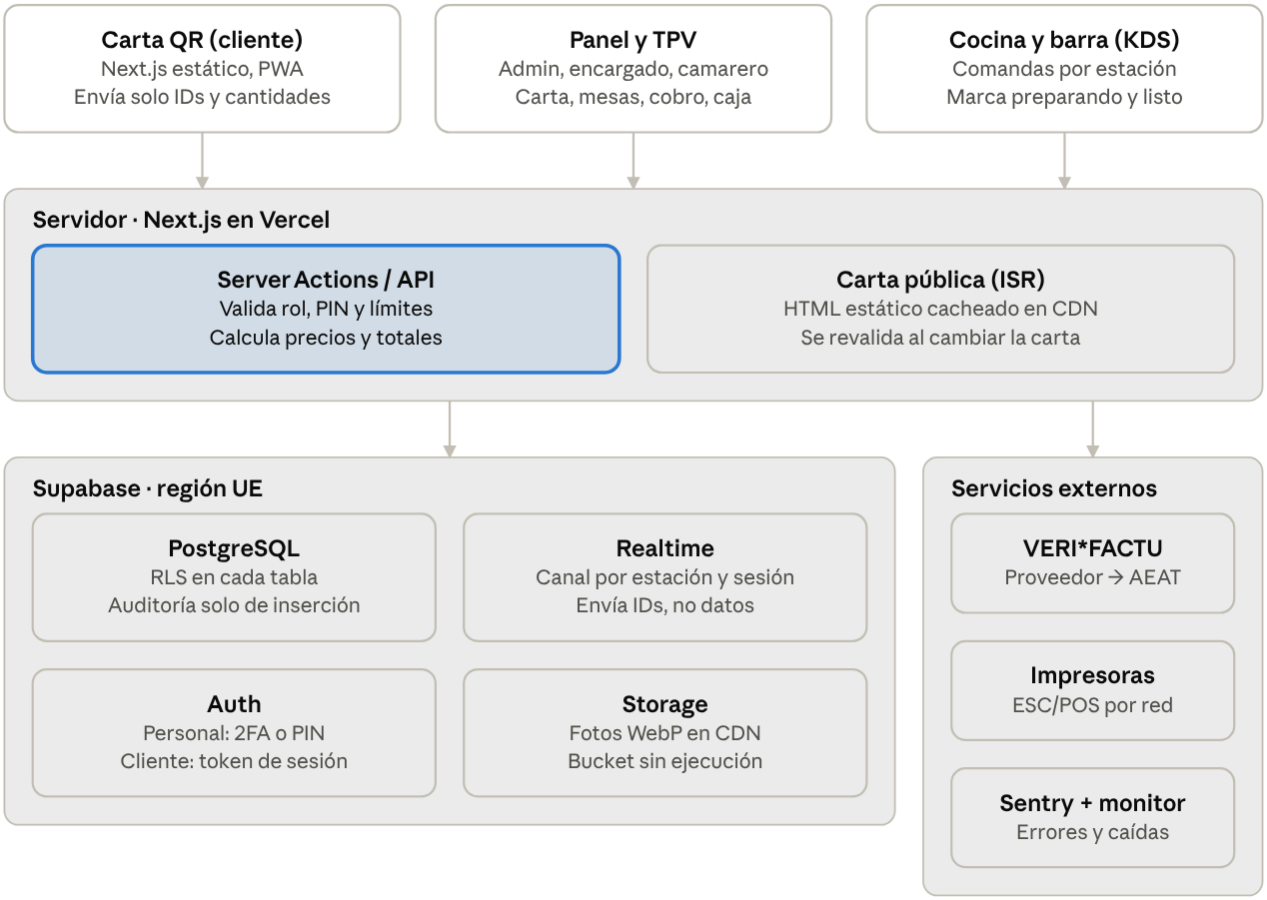
F. Cierre del día

1. El encargado abre "Cierre de caja": ve ventas por método de pago, anulaciones, descuentos e invitaciones.
2. Cuenta el efectivo, lo introduce y el sistema muestra el descuadre.
3. Al confirmar, se genera el informe Z (inmutable) y se abre el siguiente día de negocio. No se puede cerrar con mesas abiertas.

Arquitectura y stack

Una sola aplicación Next.js sirve las tres interfaces; Supabase da base de datos, autenticación, tiempo real y ficheros. Es el stack más rápido de construir para un desarrollador y escala de sobra para decenas de locales.

El servidor valida todo; nadie escribe directo en la base de datos



arquitectura · 3 frontales, servidor, Supabase, servicios externos

Toda escritura pasa por las Server Actions, que validan rol, PIN y precios. Realtime solo empuja avisos (ID y tipo) a cada pantalla; los datos se piden después y RLS filtra lo que cada uno puede ver.

Capa	Elección	Por qué
Lenguaje	TypeScript estricto	Un solo lenguaje en cliente y servidor; los tipos de la base de datos se generan solos
Frontal	Next.js (App Router) + React	Carta estática rápida, panel dinámico y Server Actions en el mismo proyecto
Interfaz	Tailwind CSS + shadcn/ui	Componentes accesibles sin una librería pesada
Validación	Zod	El mismo esquema valida en cliente y servidor

Capa	Elección	Por qué
Datos en cliente	TanStack Query	Caché, reintentos y actualizaciones optimistas
Base de datos	Supabase PostgreSQL, región UE	Transacciones, RLS, triggers, vistas materializadas
Tiempo real	Supabase Realtime	WebSockets gestionados, canales por estación y sesión
Autenticación	Supabase Auth + TOTP	2FA incluido; PIN propio para dispositivos del local
Hosting	Vercel (región `fra1` o `cdg1`)	CDN para la carta, previsualización por PR
Errores	Sentry	Trazas de cliente y servidor
Pruebas	Vitest, Playwright, k6	Unitarias, extremo a extremo y carga

Alternativa si se quiere evitar dependencia de Supabase: NestJS + PostgreSQL propio + Socket.IO. Cuesta aproximadamente un 30 % más de tiempo en las fases 0 y 2.

Modelo de datos

PostgreSQL con claves `uuid`, importes en `integer` (céntimos) y `timestamptz` en UTC. Todas las tablas llevan `restaurante_id` desde el día uno, aunque solo haya un local: añadir multi-local después sin esto obliga a migrar todo.

Tabla	Campos clave	Reglas
`restaurantes`	nombre, NIF, dirección, zona horaria, serie de tickets, ajustes (JSON)	Una fila por local
`usuarios`	email, nombre, rol, pin_hash, activo, 2fa	PIN con hash Argon2id; nunca se borran
`dispositivos`	nombre, tipo (TPV, cocina, barra), token_hash, último_acceso	El PIN de camarero solo vale en un dispositivo registrado
`categorias`	nombre (i18n JSON), orden, estación (cocina/barra), activa	Orden manual por arrastrar

Tabla	Campos clave	Reglas
`productos`	categoria_id, nombre/descripción (i18n), precio_cent, iva_pct, foto_url, disponible, activo, orden	`precio_cent >= 0`; `activo=false` en vez de borrar
`ingredientes`	nombre (i18n), alergenos[]	Alérgenos del producto = unión de los de sus ingredientes
`producto_ingredientes`	producto_id, ingrediente_id, retirable	`retirable` genera el modificador "sin X"
`grupos_modificadores`	producto_id, nombre, min, max	Ej.: "Punto de la carne", min 1, max 1
`modificadores`	grupo_id, nombre, suplemento_cent	Ej.: "+ queso", 100
`zonas`	nombre, orden	Sala, terraza, barra
`mesas`	zona_id, número, nombre, capacidad, qr_slug, pos_x, pos_y, activa	`qr_slug` único, aleatorio, regenerable
`sesiones_mesa`	mesa_id, estado, pin_hash, comensales, abierta_por, abierta_en, cerrada_en, dia_negocio_id	Índice único parcial: una sola sesión `abierta` por mesa
`tokens_cliente`	sesion_id, token_hash, creado_en, expira_en	Caduca al cerrar la sesión o a las 6 h
`comandas`	sesion_id, número, origen (cliente/personal), creada_por, estado, idempotency_key, creada_en	`idempotency_key` único por sesión
`lineas_comanda`	comanda_id, producto_id, nombre_snapshot, precio_unit_cent, iva_pct, cantidad, modificadores_snapshot (JSON), nota, estación, estado, anulada_por, motivo_anulacion	Los snapshots no se actualizan nunca

Tabla	Campos clave	Reglas
`ajustes_cuenta`	sesion_id, tipo (descuento/invitación), importe_cent o pct, motivo, autorizado_por	Con límite por rol
`pagos`	sesion_id, método, importe_cent, propina_cent, recibido_cent, referencia_tpv, creado_por	Suma de pagos = total para cerrar
`tickets`	sesion_id, serie, número, fecha, base/iva por tipo, total, hash, hash_anterior, estado_envio_aeat	Número correlativo sin huecos por serie
`dias_negocio`	abierto_en, cerrado_en, efectivo_esperado, efectivo_contado, informe_z (JSON)	Inmutable al cerrar
`eventos_auditoria`	actor, acción, entidad, entidad_id, antes (JSON), después (JSON), ip, creado_en	Solo `INSERT`; sin `UPDATE` / `DELETE`
`avisos`	sesion_id, tipo (llamar/cuenta), atendido_por, atendido_en	Para los botones del cliente

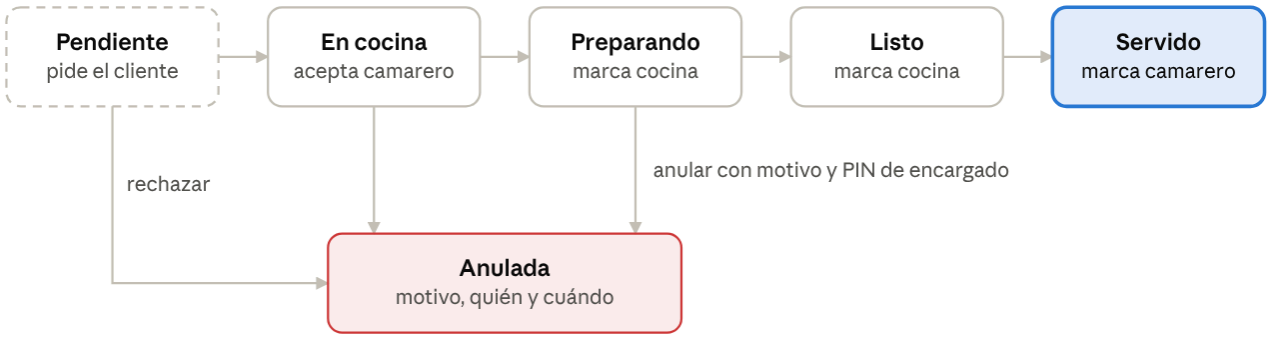
Reglas de integridad que van en la base de datos, no solo en el código

- `REVOKE UPDATE, DELETE` sobre `eventos_auditoria`, `tickets` y `dias_negocio` cerrados; un trigger rechaza cambios en `lineas_comanda` salvo estado y anulación.
- Trigger de auditoría en productos, precios, mesas, usuarios, anulaciones, descuentos y pagos.
- El total de la sesión se calcula con una vista (`v_total_sesion`), nunca se guarda a mano.
- Numeración de tickets con una tabla contador bloqueada con `SELECT … FOR UPDATE` dentro de la transacción de cobro.
- Índices: `(sesion_id)` en comandas y pagos, `(estado, estacion)` en lineas_comanda, `(restaurante_id, creada_en)` para informes, `qr_slug` único.
- Migraciones versionadas en el repositorio; nunca cambios a mano en producción.

Estados de la comanda

El estado vive en cada línea, no en la comanda: la bebida puede estar servida mientras el segundo plato se prepara. El estado de la comanda se deriva del de sus líneas.

Cada plato avanza en un solo sentido y nunca se borra



Borde discontinuo: Pendiente solo existe con la aceptación manual activa; si no, el pedido entra directo en cocina.
Listo y Servido no se anulan: se compensan con una invitación en la cuenta, que queda auditada.

estados de una línea de comanda · 5 pasos y anulación

El servidor rechaza cualquier transición que no esté en el diagrama (por ejemplo, de Servido a Preparando) y guarda la hora de cada cambio. Esas horas dan el tiempo medio de cocina en los informes.

Seguridad

El mayor riesgo no es un hacker sofisticado: es alguien que pide desde fuera del local, un precio manipulado desde el móvil o un empleado que borra ventas. Cada amenaza tiene su defensa concreta.

Amenaza	Defensa	Fase
Foto del QR para pedir desde casa	El QR solo abre la carta; pedir exige sesión abierta por el camarero + PIN de 4 dígitos. Token caduca al cerrar la mesa	2
Adivinar el PIN	5 intentos por dispositivo y 20 por sesión cada 15 min; luego bloqueo y aviso al camarero	2
Enumerar mesas cambiando la URL	`qr_slug` aleatorio de 22 caracteres, nunca el número de mesa; respuestas idénticas para slug inexistente o inactivo	1

Amenaza	Defensa	Fase
Precio manipulado desde el móvil	El cliente envía solo IDs y cantidades; el servidor busca precios, valida modificadores (min/max, pertenencia al producto) y calcula	2
Pedido duplicado por doble toque o mala red	`idempotency_key` única por sesión; el segundo envío devuelve la misma comanda	2
Spam de pedidos o de "llamar camarero"	Límite de 1 comanda cada 20 s y 10 por sesión y hora por dispositivo; máximo 30 unidades por línea; opción de aceptación manual	2
Ver pedidos de otra mesa	RLS: el token de cliente solo lee filas de su `sesion_id`; pruebas automáticas que lo verifican	2
Empleado borra o rebaja ventas	Sin borrados; anulaciones y descuentos exigen motivo y, por encima de un límite, PIN de encargado; todo en `eventos_auditoria`	2–3
Alterar tickets ya emitidos	Tickets encadenados por hash (SHA-256 del anterior + datos), sin `UPDATE`; rectificativas en vez de cambios	4
Robo de cuenta del admin	2FA TOTP obligatorio, contraseña mínimo 12 caracteres comprobada contra filtradas, sesiones de 12 h, cierre remoto de sesiones	0
Tablet de cocina robada	Dispositivos registrados con token revocable desde el panel; el rol cocina no ve importes ni datos de pago	2
XSS en notas o nombres de productos	React escapa por defecto; prohibido `dangerouslySetInnerHTML`; CSP estricta; notas limitadas a 140 caracteres de texto plano	1
Subida de imágenes maliciosas	Solo JPEG/PNG/WebP hasta 5 MB, se re-codifican en servidor y se sirven desde un bucket sin ejecución	1
Claves filtradas	La clave `service_role` de Supabase solo en el servidor; secretos en variables de entorno; escaneo de secretos en CI	0
Datos de tarjeta	El sistema nunca toca números de tarjeta: el datáfono es independiente y solo se guarda el importe y la referencia	3

Cabeceras y configuración obligatorias

- HTTPS siempre, HSTS, `Content-Security-Policy` sin `unsafe-inline` para scripts, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`.
- Cookies `HttpOnly`, `Secure`, `SameSite=Lax`; protección CSRF en las mutaciones del panel.
- RLS activado en todas las tablas sin excepción; una prueba de CI falla si alguna tabla nueva no lo tiene.
- Dependencias: `npm audit` y Dependabot semanales; actualizaciones de seguridad en menos de 7 días.
- Auditoría externa ligera (OWASP ASVS nivel 1) antes de abrir a un segundo restaurante.

Rendimiento y optimización

El cliente escanea con datos móviles malos y la cocina no puede esperar: la carta debe abrir en menos de 2 s en 4G y una comanda debe llegar a cocina en menos de 1 s.

Métrica	Objetivo	Cómo se mide
Carga de la carta (LCP, móvil 4G)	< 2,0 s	Lighthouse en CI + Vercel Speed Insights
JavaScript de la carta pública	< 120 KB gzip	`next build` + límite en CI
Pedido enviado → visible en cocina	< 1 s (p95)	Marca de tiempo en servidor vs. recepción en pantalla
Respuesta de la API (p95)	< 300 ms	Logs/APM
Pantalla de cocina con 50 comandas abiertas	Sin tirones (60 fps)	Prueba manual en la tablet real
Carga sostenida	100 mesas, 2 comandas/min cada una, sin errores	Prueba con k6 antes de producción

Carta pública

- Renderizado estático/ISR por restaurante; se revalida solo cuando el admin cambia la carta (`revalidateTag`). La disponibilidad (agotado) llega aparte por tiempo real.
- Imágenes en WebP/AVIF, varios tamaños (`srcset`), carga diferida, ancho máximo 800 px. Una foto no debe pasar de 80 KB.

- Sin librerías pesadas en la carta: nada de librerías de gráficos, editores ni iconos completos.
- Carrito en el estado del cliente y guardado en `localStorage`: si se recarga la página, no se pierde.
- PWA ligera: cachea la carta y las imágenes con un service worker; si se va la red, muestra la carta y avisa de que no se puede pedir.

Tiempo real

- Un canal por restaurante y estación para el personal; un canal por sesión para el cliente. Nunca un canal global.
- Los eventos llevan solo el ID y el tipo; la pantalla pide los datos que le faltan. Así nadie recibe datos que no le tocan.
- Al reconectar, la pantalla pide todo lo cambiado desde su última marca de tiempo (no confía en no haberse perdido nada).
- La cocina tiene sondeo de respaldo cada 15 s si el WebSocket cae, con aviso visible "Sin conexión".

Base de datos

- Los índices de la sección Modelo de datos, revisados con `EXPLAIN ANALYZE` en las 5 consultas más usadas.
- Pool de conexiones (Supavisor/PgBouncer) en modo transacción desde las funciones serverless.
- Los informes históricos leen de vistas materializadas por día que se refrescan al cerrar caja; nunca recorren todas las líneas en cada consulta.
- Paginación por cursor (`creada_en` , `id`) en el histórico, no `OFFSET` .

Panel y TPV

- Actualizaciones optimistas al cambiar estados, con reversión si el servidor falla.
- Listas largas (histórico, productos) virtualizadas.
- La interfaz de camarero se diseña para táctil: botones de al menos 48 px y como máximo 3 toques para añadir un producto.

Legal (España)

El sistema emite tickets, así que es software de facturación y debe cumplir VERI*FACTU. La fecha está en movimiento: hay que diseñarlo desde la fase 3 y activarlo antes de la fecha que esté en el BOE en ese momento.

VERI*FACTU (RD 1007/2023 y Orden HAC/1177/2024)

- Calendario en el BOE (RDL 15/2025): 1 de enero de 2027 para sociedades y 1 de julio de 2027 para autónomos.
- Según prensa de octubre de 2026, Hacienda ha anunciado un nuevo aplazamiento hasta octubre de 2028 que aún no estaría en el BOE. Pendiente de confirmar con el gestor y en la sede de la AEAT.
- Requisitos técnicos: cada ticket genera un registro de facturación con huella SHA-256 encadenada al anterior, un QR de cotejo impreso en el ticket, la leyenda "VERI*FACTU" y envío automático a la AEAT. Las anulaciones se hacen con registros de anulación o facturas rectificativas, nunca borrando.
- Recomendación: no implementar el envío a la AEAT a mano. Integrar un proveedor ya certificado por API y que el sistema guarde el registro, la huella y el estado de envío. Coste orientativo: de 10 a 30 € al mes por local (comparar 2–3 proveedores en la fase 4).
- Declaración responsable del fabricante del software: si el restaurante lo usa con su NIF y lo desarrolla un tercero, el desarrollador debe emitirla.
- En País Vasco y Navarra aplica TicketBAI en lugar de VERI*FACTU.

Alérgenos (Reglamento UE 1169/2011 y RD 126/2015)

- La carta debe informar de los 14 alérgenos: gluten, crustáceos, huevo, pescado, cacahuete, soja, lácteos, frutos de cáscara, apio, mostaza, sésamo, sulfitos, altramuces y moluscos.
- Se calculan a partir de los ingredientes, pero el admin puede añadir "puede contener trazas de" a mano.
- Filtro en la carta: "Ocultar platos con…". Aviso fijo: "Consulte al personal si tiene alergias".

Datos personales (RGPD y LOPDGDD)

- El cliente no deja datos personales: ni nombre, ni teléfono, ni email. El token es anónimo y caduca. Así se evita casi todo el RGPD.
- Banner de cookies solo si se añade analítica de terceros; la cookie de sesión es técnica y no lo necesita.
- Para los empleados: contrato de encargado de tratamiento con el proveedor de hosting, servidores en la UE y registro de actividades de tratamiento.
- Conservación: tickets y registros de facturación 4 años como mínimo (prescripción tributaria); 6 años recomendados por el Código de Comercio.

Otros

- Precios con IVA incluido en la carta (obligatorio para consumidores). IVA de hostelería: 10 % en comida y bebida, salvo excepciones que el admin configura por producto.

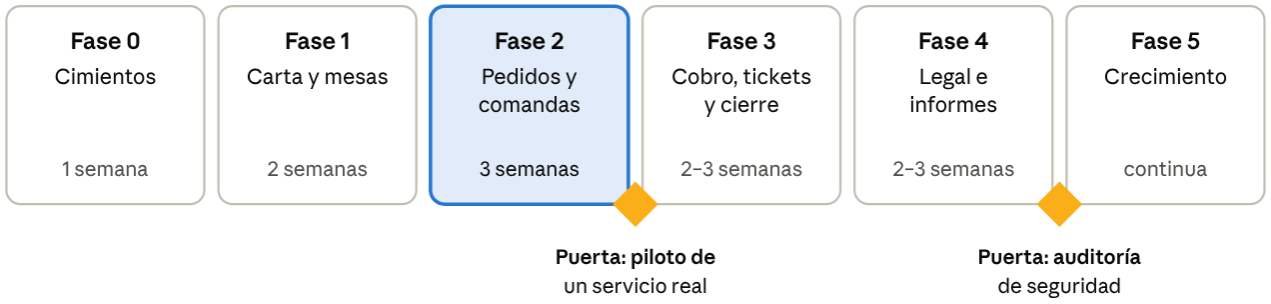
- Ticket con NIF, razón social, número correlativo, fecha, desglose de IVA y total.

Fuentes (resultados de búsqueda; no se pudieron abrir desde este entorno): infoautonomos, Infocop sobre el aplazamiento a 2028, Ecosistema Startup, Billin sobre el RDL 15/2025.

Fases

De la fase 0 a la 4 hay entre 10 y 12 semanas para un desarrollador a tiempo completo. La fase 2 es el núcleo del producto y la única que exige un piloto en el restaurante antes de seguir.

De 10 a 12 semanas hasta un TPV legal; el piloto tras la fase 2 decide



Cada fase cierra con sus criterios de aceptación revisados por el auditor; los rombos son puertas adicionales.

hoja de ruta · 6 fases, 2 puertas

Las bandas no están a escala. La fase 4 debe terminar antes de la fecha de VERI*FACTU que aplique al local.

Fases en detalle

Una fase está terminada cuando se cumplen todos sus criterios de aceptación en el entorno de staging y el auditor los ha revisado. Las semanas son una estimación para un desarrollador a tiempo completo.

Fase 0 — Cimientos (1 semana)

Objetivo: un esqueleto desplegado con login seguro, base de datos y CI, sin funcionalidad de negocio.

- [ ] Repositorio nuevo (monorepo con `apps/web` y `packages/db`), TypeScript estricto, ESLint y Prettier.
- [ ] Entornos `local`, `staging` y `produccion`, cada uno con su proyecto de Supabase y sus variables.

- [ ] Esquema base: `restaurantes`, `usuarios`, `dispositivos`, `eventos_auditoria`, con migraciones versionadas.
- [ ] Login de admin con email + contraseña + 2FA TOTP; login por PIN en dispositivo registrado.
- [ ] RLS activado en todas las tablas y prueba de CI que falla si falta.
- [ ] CI en GitHub Actions: lint, typecheck, pruebas, build y escaneo de secretos en cada PR.
- [ ] Cabeceras de seguridad (CSP, HSTS) y Sentry conectado.

Criterios de aceptación:

1. Un usuario sin 2FA no puede entrar al panel de admin.
2. Una consulta con la clave anónima a cualquier tabla devuelve 0 filas.
3. Un PR con un error de tipos no se puede fusionar.

Fase 1 — Carta y mesas (2 semanas)

Objetivo: el admin monta la carta completa y las mesas; el cliente puede verla por QR, sin pedir.

- [ ] CRUD de categorías (con orden por arrastrar y estación), productos, ingredientes, alérgenos y grupos de modificadores.
- [ ] Subida de fotos con re-codificación a WebP y varios tamaños.
- [ ] Desactivar en vez de borrar; vista de "productos inactivos".
- [ ] Interruptor de "agotado" en un toque.
- [ ] CRUD de zonas y mesas; generación de `qr_slug`; descarga de QR individual y hoja A4 imprimible; botón de regenerar.
- [ ] Carta pública `/m/<qr_slug>`: categorías, fotos, precios con IVA, alérgenos con iconos, filtro por alérgeno.
- [ ] Auditoría de cambios de precio y de carta.

Criterios de aceptación:

1. Cambiar un precio en el panel se ve en la carta pública en menos de 10 s sin desplegar.
2. Un `qr_slug` regenerado deja de funcionar al instante.
3. La carta con 80 productos y fotos carga en menos de 2 s en un móvil con 4G simulado (Lighthouse ≥ 90 en rendimiento).
4. Los alérgenos de un plato coinciden con la unión de los de sus ingredientes.

Fase 2 — Pedidos y comandas (3 semanas)

Objetivo: el flujo completo de pedir, de la mesa a la cocina, en tiempo real. Es el corazón del producto.

- [ ] Plano de mesas para el camarero con estados por color (libre, ocupada, pidiendo, cuenta pedida).
- [ ] Abrir sesión con comensales y PIN; restricción de una sesión abierta por mesa.
- [ ] Canje de PIN por token de cliente (cookie `HttpOnly`), límites de intentos.
- [ ] Carrito con modificadores, notas y persistencia en `localStorage`.
- [ ] Endpoint de crear comanda: validación, precio en servidor, snapshots, `idempotency_key`, transacción única.
- [ ] Pedido desde el TPV del camarero (mismo endpoint, origen "personal").
- [ ] Tiempo real: panel, pantalla de cocina y barra, y estado en el móvil del cliente; reconexión con recuperación.
- [ ] Pantalla de cocina (KDS): columnas por estado, tiempo desde que entró (en rojo pasados 15 min), sonido, filtros por estación.
- [ ] Anulación de línea con motivo y PIN de encargado.
- [ ] Botones "Llamar camarero" y "Pedir la cuenta" con aviso en el panel.
- [ ] Total de la sesión en vivo en el panel de la mesa.
- [ ] Límites anti-spam y modo "aceptación manual".

Criterios de aceptación:

1. Un pedido aparece en cocina en menos de 1 s (p95) desde que el cliente confirma.
2. Pulsar "Pedir" dos veces seguidas crea una sola comanda.
3. Un token de una mesa no puede leer ni crear comandas en otra (prueba automatizada).
4. Con la mesa cerrada, el mismo móvil ya no puede pedir.
5. Manipular el precio en la petición no cambia lo que se cobra.
6. Si la tablet de cocina pierde la red 2 minutos, al volver muestra todas las comandas que entraron.

Puerta: piloto de un servicio completo (comida o cena) en el restaurante real, con el sistema en paralelo al método actual. No se pasa a la fase 3 sin ese piloto.

Fase 3 — Cobro, tickets y cierre (2–3 semanas)

Objetivo: cerrar el ciclo del dinero con un registro fiable.

- [ ] Pantalla de cobro: total, desglose de IVA, descuentos e invitaciones con motivo y límite por rol.
- [ ] Dividir la cuenta a partes iguales o por productos; varios pagos por sesión.

- [ ] Métodos: efectivo (con cambio), tarjeta (datáfono externo, se anota la referencia) y mixto; propina.
- [ ] Ticket (factura simplificada) con numeración correlativa sin huecos, en pantalla, PDF e impresora térmica.
- [ ] Impresión de comandas en impresoras de cocina y barra (ESC/POS por red) como alternativa a la pantalla.
- [ ] Cierre de la sesión: invalida tokens y libera la mesa.
- [ ] Reabrir cuenta (solo admin) generando ticket rectificativo.
- [ ] Días de negocio y cierre de caja con arqueo e informe Z.
- [ ] Histórico: buscar por fecha, mesa, camarero o importe; ver cada sesión con sus comandas y horas; exportar CSV.

Criterios de aceptación:

1. La suma de los informes Z de un mes coincide al céntimo con la suma de pagos de ese mes.
2. Dos cobros simultáneos nunca generan el mismo número de ticket (prueba de concurrencia).
3. Un ticket emitido no se puede modificar por ninguna vía, ni siquiera con acceso de admin desde la API.
4. No se puede cerrar caja con mesas abiertas.

Fase 4 — Legal e informes (2–3 semanas)

Objetivo: cumplir VERI*FACTU y dar al dueño información para decidir.

- [ ] Elegir proveedor VERI*FACTU certificado e integrarlo: registro, huella encadenada, QR en el ticket, envío y reintentos con cola.
- [ ] Panel de estado de envíos a la AEAT con alertas si falla.
- [ ] Informes: ventas por día y hora, por producto y categoría, por camarero, ticket medio, tiempo medio de cocina, anulaciones.
- [ ] Vistas materializadas por día para que los informes sean rápidos.
- [ ] Exportación para el gestor (CSV/Excel con desglose de IVA).

Criterios de aceptación:

1. Cada ticket tiene su QR de cotejo y su estado de envío; un envío fallido se reintenta solo.
2. El informe de un año de datos carga en menos de 2 s.

Fase 5 — Crecimiento (continua, por prioridad)

- Carta en varios idiomas (el campo i18n ya existe desde la fase 1).
- Stock por ingrediente con descuento automático y aviso de mínimos.
- Pago desde el móvil del cliente (Stripe o Redsys) y propina online.
- Multi-local con panel de grupo.
- Modo offline del TPV con cola de sincronización.
- Integración con datáfono para enviar el importe automáticamente.

Calidad y operación

Un TPV que se cae un sábado a las 21:30 es un restaurante parado. Las pruebas y la monitorización son parte del producto, no un extra.

Pruebas

Tipo	Herramienta	Qué cubre como mínimo
Unitarias	Vitest	Cálculo de totales, IVA, redondeos, división de cuenta, validación de modificadores (cobertura ≥ 90 % en `packages/core`)
Base de datos	pgTAP o pruebas SQL en CI	Políticas RLS por rol, triggers de inmutabilidad, restricción de una sesión por mesa
Integración	Vitest + Supabase local	Endpoints de comanda, cobro, cierre de caja, idempotencia, concurrencia de tickets
Extremo a extremo	Playwright	Flujo completo: abrir mesa → escanear → PIN → pedir → cocina → cobrar → cerrar caja
Carga	k6	100 mesas pidiendo a la vez durante 30 min
Rendimiento	Lighthouse CI	Carta pública ≥ 90 en móvil

CI/CD

- Cada PR: lint, typecheck, unitarias, base de datos, integración, build y Lighthouse. Playwright en cada PR contra un despliegue de vista previa.
- Rama `main` protegida: requiere CI verde y revisión.
- Despliegue a staging automático al fusionar; a producción con aprobación manual.
- Prohibido desplegar a producción en horario de servicio (12:00–16:30 y 20:00–00:30); regla en el flujo de despliegue.

- Migraciones compatibles hacia atrás (añadir columna → desplegar código → quitar lo viejo), para poder revertir el código sin romper datos.

Observabilidad

- Sentry para errores en cliente y servidor, con avisos al móvil para errores nuevos en producción.
- Logs estructurados (JSON) con `restaurante_id`, `sesion_id` y `request_id`; nunca PIN, tokens ni datos personales.
- Monitor externo (UptimeRobot o Better Stack) cada minuto sobre la carta, la API y el canal de tiempo real.
- Alertas: tasa de error > 1 % en 5 min, latencia de comanda p95 > 2 s, envío a la AEAT fallido > 30 min.

Copias de seguridad y recuperación

- Supabase con Point-in-Time Recovery (plan Pro o superior) y retención mínima de 7 días.
- Volcado diario cifrado (`pg_dump`) a un almacenamiento distinto del proveedor, con retención de 6 años para los datos fiscales.
- Simulacro de restauración una vez al mes en staging; si no se ha probado, no hay copia.
- Objetivos: pérdida máxima de datos 5 min (RPO), vuelta al servicio en menos de 1 h (RTO).
- Plan B en sala: si todo falla, el camarero toma comandas en papel y las mete después; el sistema permite registrar comandas con hora anterior (marcadas y auditadas).

Coste orientativo mensual (un local)

Concepto	Coste
Vercel Pro	~20 €
Supabase Pro (con PITR)	~25–125 €
Sentry y monitor de disponibilidad	0–30 €
Proveedor VERI*FACTU	~10–30 €
Dominio	~1 €

Precios aproximados de memoria; confirmarlos al contratar.

Reglas de trabajo para el desarrollador

El auditor revisa cada PR contra esta lista; un PR que no la cumple se devuelve sin revisar el resto.

Cómo se entrega

1. Una rama y un PR por tarea de la checklist; PRs de menos de 400 líneas cambiadas cuando sea posible.
2. Descripción del PR: qué hace, cómo probarlo, capturas si toca interfaz, y qué criterio de aceptación cubre.
3. Demo grabada (1–2 min) al cerrar cada fase.
4. Las decisiones que se aparten de este documento se anotan como comentario en él antes de programarlas.

Definición de terminado (cada PR)

- [ ] CI en verde: lint, tipos, pruebas, build.
- [ ] Pruebas nuevas para la lógica nueva; las de RLS si toca tablas.
- [ ] Ninguna tabla nueva sin RLS ni sin `restaurante_id`.
- [ ] Ningún importe en `float`; ningún precio que venga del cliente.
- [ ] Ningún `DELETE` sobre datos de negocio; cambios relevantes pasan por auditoría.
- [ ] Sin secretos en el código; variables nuevas documentadas en `.env.example`.
- [ ] Funciona en móvil (375 px) y en tablet horizontal.
- [ ] Migración incluida y reversible si hay cambios de esquema.
- [ ] Textos de interfaz en español y preparados para traducción.

Prohibido sin aprobación del auditor

- Añadir dependencias de más de 50 KB a la carta pública.
- Desactivar RLS, saltarse una prueba o marcarla como `skip`.
- Cambios manuales en la base de datos de producción.
- Implementar criptografía o el envío a la AEAT a mano.

Preguntas abiertas para el dueño

Estas respuestas cambian el diseño; las de la fase 2 se necesitan antes de empezarla.

- [ ] ¿El local factura como sociedad o como autónomo? Define la fecha de VERI*FACTU.
- [ ] ¿Está en País Vasco o Navarra? (TicketBAI en lugar de VERI*FACTU.)
- [ ] ¿Cuántas mesas y cuántos camareros en un sábado lleno?
- [ ] ¿La cocina prefiere pantalla, impresora o las dos? ¿Qué impresoras hay ya?

- [ ] ¿Los pedidos del cliente entran directos a cocina o el camarero los acepta antes?
- [ ] ¿Se cobra en mesa con datáfono portátil o en barra?
- [ ] ¿Hay menú del día o menús cerrados con varios platos a elegir? (Requiere un tipo de producto "menú".)
- [ ] ¿Hay precios distintos por zona u horario (terraza más cara, happy hour)?
- [ ] ¿Idiomas necesarios para la carta?
- [ ] ¿Quién será el dueño legal del software y de los datos si lo desarrolla un tercero?